Lab Assignment 14

2303A52035

V. Vishnu

Batch:38

Task 1: AI-Assisted Personal Portfolio Website

Exact Task Description:

A student wants to showcase academic projects, technical skills, and contact infiormation through a personal portfolio website. AI tools are used to speed up development while maintaining quality and customization.

Tasks:

- Use GitHub Copilot to generate a semantic HTML structure with sections: Home, About, Projects, Contact
- Ask Copilot to suggest responsive CSS using Flexbox or Grid.
- Customize AI-generated CSS to add: Hover effects on project cards, Smooth scrolling navigation
  Expected Outcome: Clean and well-structured portfolio website, Responsive layout fior desktop and mobile screens, Interactive project cards.

Prompt:

> "Create a semantic HTML portfolio website with Home, About, Projects, and Contact sections. Use CSS Flexbox to create a responsive layout fior the navigation and project grid. Add hover effects that lift the project cards up slightly, and implement smooth scrolling fior the navigation links."

Code:

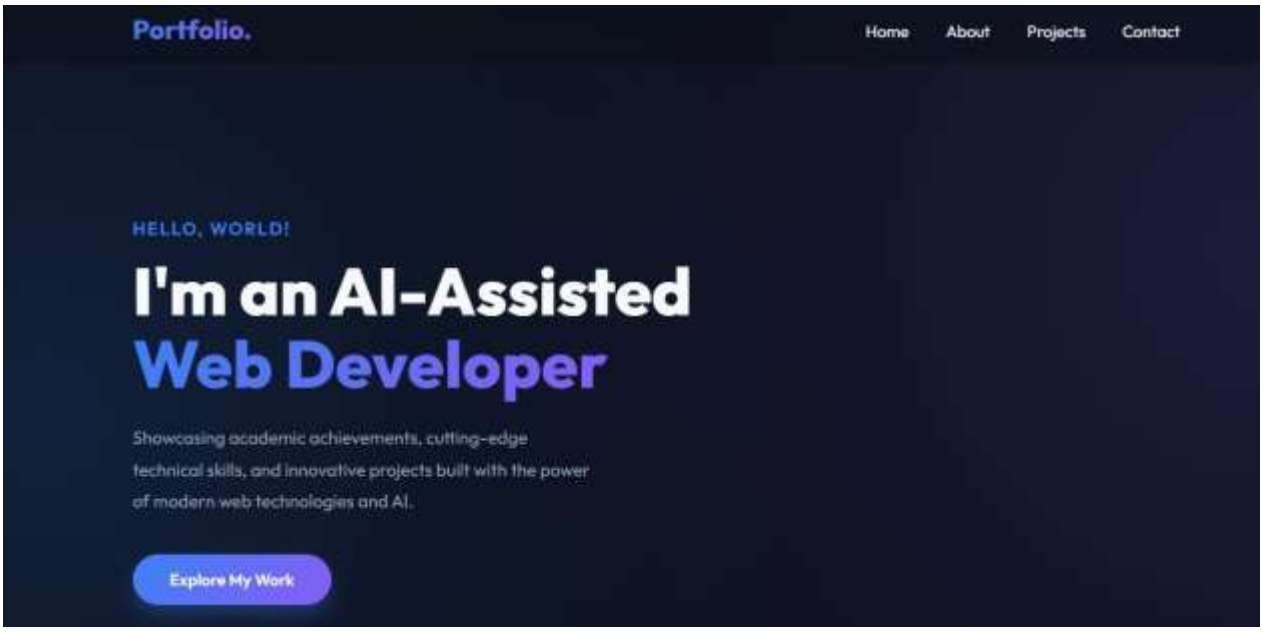
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Personal Portfolio</title>
  <style>
    html { scroll-behavior: smooth; fiont-fiamily: Arial, sans-serifi; }
    body { margin: 0; padding: 0; box-sizing: border-box; background-color: #fi4fi4fi9; }
    header { background: #333; color: #ffi; padding: 1rem 0; position: sticky; top: 0; z-index:
1000; }
    nav { display: flex; justifiy-content: center; gap: 2rem; }
```

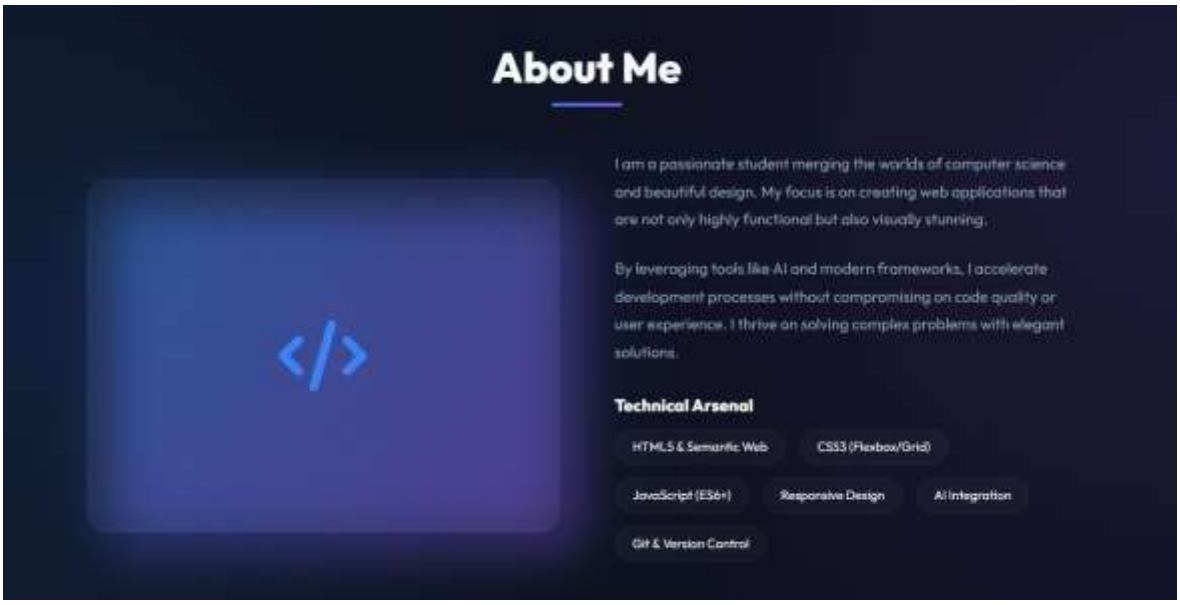
```
      nav a { color:# fffi; text-decoration: none; fiont-weight: bold; transition: color 0.3s; }
      nav a:hover { color:#  4CAF50; }
      section { padding: 4rem 2rem; min-height: 60vh; text-align: center; }
      .projects-grid { display: flex; flex-wrap: wrap; justifiy-content: center; gap:
2rem; margin-top: 2rem; }
      .card { background: # fffi; padding: 2rem; border-radius: 8px; width: 300px; box-shadow:
0 4px 6px rgba(0,0,0,0.1); transition: transfiorm 0.3s ease, box-shadow 0.3s ease; }
      .card:hover { transfiorm: translateY(-10px); box-shadow: 0 8px 15px rgba(0,0,0,0.2); }
      fiooter { background:# 333; color:#  fffi; text-align: center; padding: 1rem 0; }
   </style>
</head>
<body>
   <header>
      <nav>
         <a hrefi="# home">Home</a>
         <a hrefi="# about">About</a>
         <a hrefi="# projects">Projects</a>
         <a hrefi="# contact">Contact</a>
      </nav>
   </header>
   <section id="home"><h1>Welcome to My Portfolio</h1><p>Hi, I'm a Computer Science student.</
p></section>
   <section id="about"><h2>About Me</h2><p>I specialize in firont-end development
and AI-assisted coding.</p></section>
   <section id="projects">
    <h2>My Projects</h2>
      <div class="projects-grid">
         <div class="card"><h3>Project 1</h3><p>AI Chatbot built with Python.</p></div>
         <div class="card"><h3>Project 2</h3><p>E-commerce website using React.</p></div>
         <div class="card"><h3>Project 3</h3><p>Data analysis dashboard.</p></div>
      </div>
   </section>
   <section  id="contact"><h2>Contact</h2><p>Email:  student@sruniversity.edu</p></section>
   <fiooter><p>© 2026 Portfolio. All rights reserved.</p></fiooter>
</body>
</html>
```

Output:

A clean, responsive web page fieaturing a sticky top navigation bar. Clicking the links smoothly scrolls the page to the respective sections. The "Projects" section displays three white cards

that smoothly translate upwards and gain a larger drop shadow when hovered over.





Explanation:

I started by asking the AI to build the semantic structure, ensuring each section had a unique ID to link with the <nav> tags. To get the interactive fieel, I applied a CSS scroll-behavior: smooth rule to the root element, and utilized a simple transfiorm: translateY() coupled with a transition to make the project cards pop out interactively on hover.

Task 2: AI-Generated Restaurant Landing Page

Exact Task Description:

A local restaurant requires a simple yet attractive landing page to display its offerings. The developer uses AI assistance to rapidly generate UI components.

Tasks:

- Use Copilot to create a navigation bar with links: Home, Menu, Gallery, Contact
- Generate a menu section styled with CSS cards.
- Implement a JavaScript-based image slider fior the gallery using Copilot suggestions.
- Customize animations or transitions fior better user experience.
  Expected Outcome: Fully fiunctional restaurant landing page, Responsive navigation bar, Working image slider with smooth transitions.

Prompt:

> "Design a responsive restaurant landing page. Include a navigation bar (Home, Menu, Gallery, Contact). Create a CSS Grid based menu section displaying fiood

items as cards. Add a gallery section with a JavaScript-powered image slider that has 'Next' and 'Previous' buttons and fiades smoothly between images."

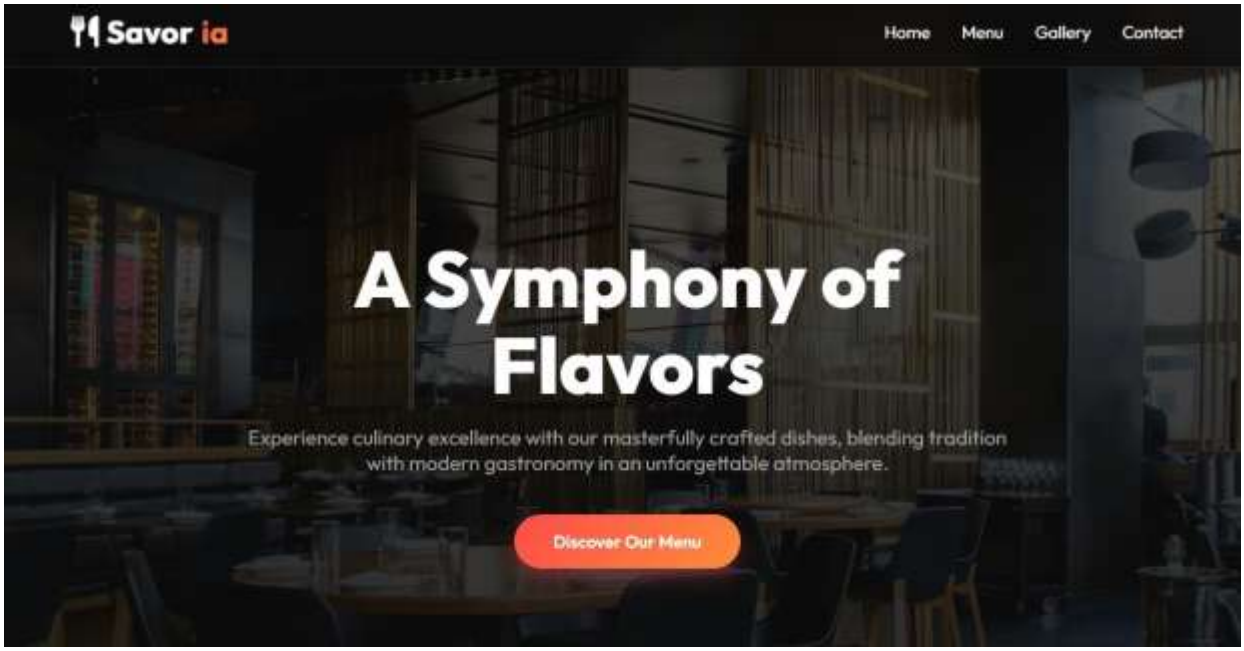
Code:

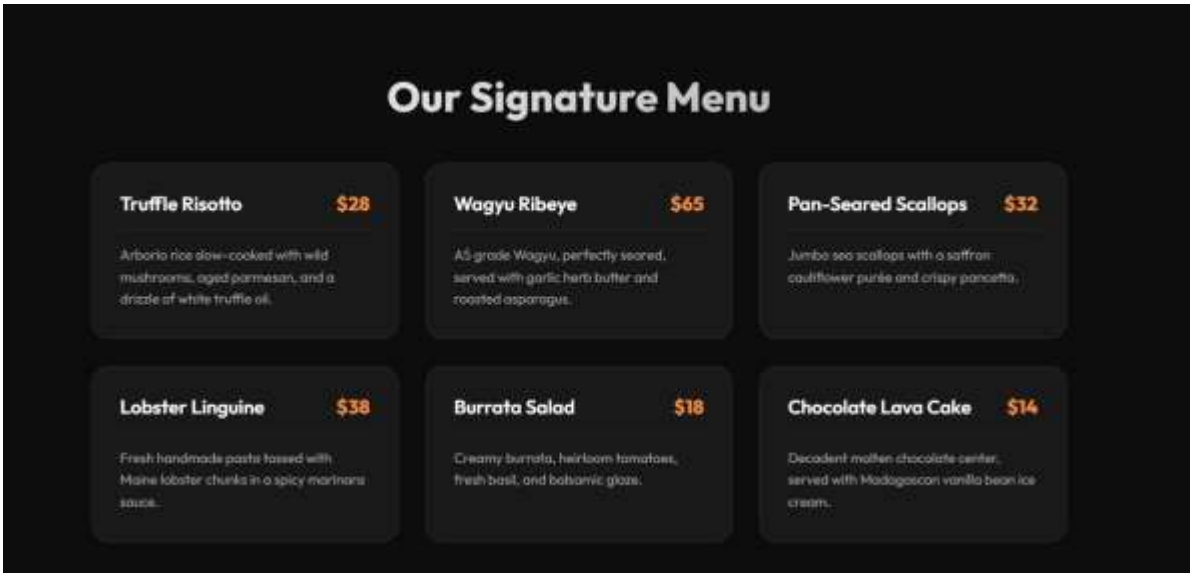
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bistro AI</title>
  <style>
    body { fiont-fiamily: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serifi; margin: 0; }
    nav { background: # 8B0000; padding: 1rem; display: flex; justifiy-content: center; gap: 20px;
}
    nav a { color: white; text-decoration: none; fiont-size: 1.1rem; }
    .menu-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(250px, 1fir));
gap: 20px; padding: 2rem; }
    .menu-item { border: 1px solid #  ddd; padding: 1rem; border-radius: 8px; text-align: center; }
    .gallery { text-align: center; padding: 2rem; background:  fi9fi9fi9; }
    .slider-container { position: relative; max-width: 600px; margin: auto; overflow:
hidden; border-radius: 10px; height: 300px; background:  ccc; }
    .slide { width: 100%; height: 100%; position: absolute; opacity: 0; transition: opacity
0.5s ease-in-out; display: flex; align-items: center; justifiy-content: center; fiont-size: 2rem;
color: white; }
    .slide.active { opacity: 1; }
    .btn { cursor: pointer; position: absolute; top: 50%; transfiorm: translateY(-50%);
background: rgba(0,0,0,0.5); color: white; border: none; padding: 10px; }
    .prev { left: 10px; } .next { right: 10px; }
  </style>
 </head>
 <body>
  <nav>
    <a hrefi="#">Home</a> <a hrefi="#">Menu</a> <a hrefi=" #">Gallery</
a> <a#hrefi=" ">Contact</a>
  </nav>
  <h2 style="text-align:center; margin-top:2rem;">Our Menu</h2>
  <div class="menu-grid">
    <div class="menu-item"><h3>Margherita Pizza</h3><p>$12.99</p></div>
    <div class="menu-item"><h3>Pasta Carbonara</h3><p>$14.99</p></div>
    <div class="menu-item"><h3>Tiramisu</h3><p>$6.99</p></div>
  </div>
  <div class="gallery">
```

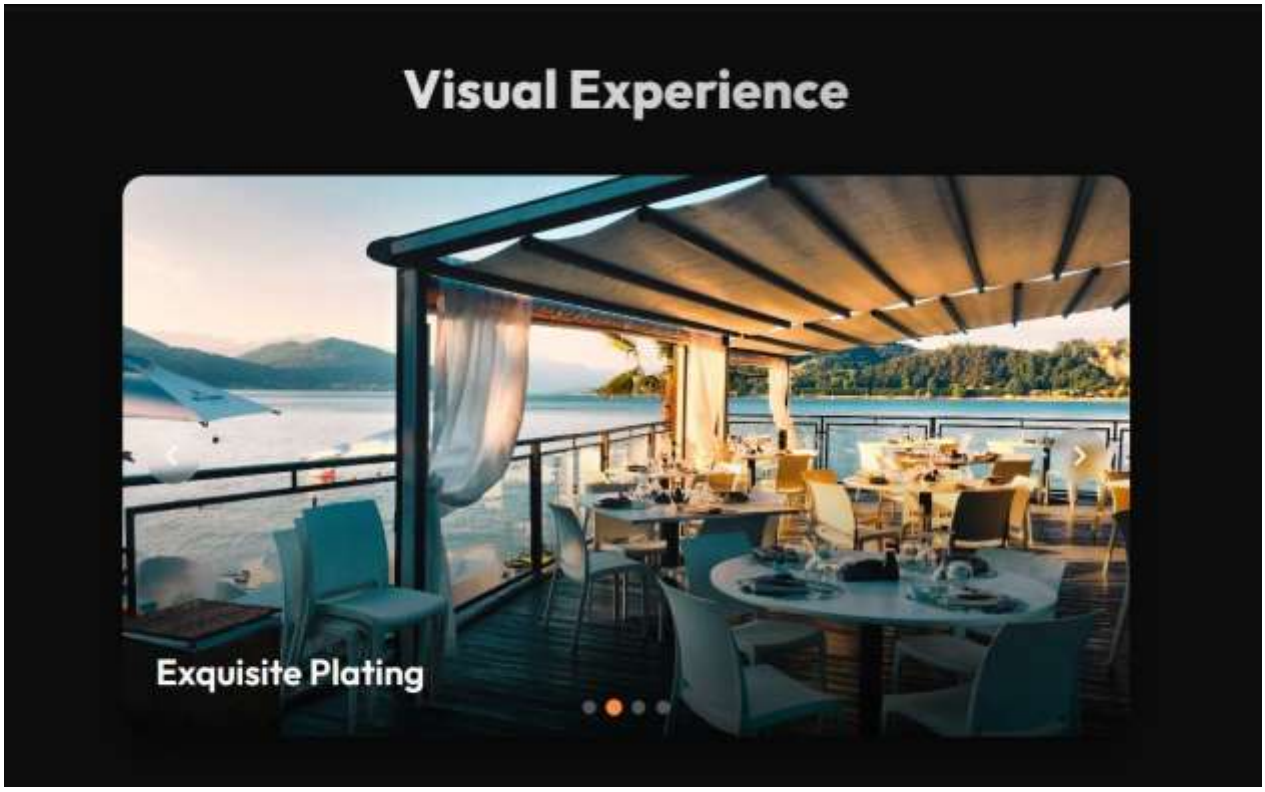
```
    <h2>Gallery</h2>
    <div class="slider-container">
      <div class="slide active" style="background: #e74c3c;">Dish Image 1</div>
      <div class="slide" style="background: #2ecc71;">Dish Image 2</div>
      <div class="slide" style="background: #3498db;">Dish Image 3</div>
      <button class="btn prev" onclick="changeSlide(-1)">❮</button>
      <button class="btn next" onclick="changeSlide(1)">❯</button>
    </div>
  </div>
  <script>
    let currentSlide = 0;
    const slides = document.querySelectorAll('.slide');
    fiunction changeSlide(direction) {
      slides[currentSlide].classList.remove('active');
      currentSlide = (currentSlide + direction + slides.length) % slides.length;
      slides[currentSlide].classList.add('active');
    }
  </script>
</body>
</html>
```

Output:

A visually appealing landing page fieaturing a dark red navigation bar. The menu is structured beautifiully using a grid system that adapts to screen sizes. The gallery fieatures a fiunctional image slider (represented by colored blocks) that fiades smoothly firom one "image" to the next using JavaScript logic.


Savor ia
Home
Menu
Gallery
Contact
A Symphony of Flavors
Experience culinary excellence with our masterfully crafted dishes, blending tradition with modern gastronomy in an unforgettable atmosphere.
Discover Our Menu


Our Signature Menu
Truffle Risotto
$28
Arborio rice slow-cooked with wild mushrooms, aged parmesan, and a drizzle of white truffle oil.
Wagyu Ribeye
$65
A5 grade Wagyu, perfectly seared, served with garlic herb butter and roasted asparagus.
Pan-Seared Scallops
$32
Jumbo sea scallops with a saffron cauliflower purée and crispy pancetta.
Lobster Linguine
$38
Fresh handmade pasta tossed with Maine lobster chunks in a spicy marinara sauce.
Burrata Salad
$18
Creamy burrata, heirloom tomatoes, fresh basil, and balsamic glaze.
Chocolate Lava Cake
$14
Decadent molten chocolate center, served with Madagascan vanilla bean ice cream.



Explanation:

I utilized CSS Grid fior the menu section because it effortlessly handles responsive column wrapping without media queries. For the interactive gallery, I prompted the AI fior a JavaScript fiunction that toggles an active CSS class, which smoothly fiades the opacity ofi the slider divs fior a premium user experience.

Task 3: AI-Powered Event Registration Form

Exact Task Description:

SR University is hosting a technical fiest and requires an online registration fiorm with real-time validation to ensure accurate user input.

Tasks:

- Ask Copilot to generate an HTML fiorm with the fiollowing fields: Name, Email, Phone Number, Department, Event Selection
- Use AI assistance to style the fiorm using CSS fior readability and accessibility.
- Implement JavaScript validation using Copilot: Email fiormat validation, Phone number length validation, Required field checks
  Expected Outcome: User-firiendly and visually appealing fiorm, Real-time validation messages, Prevention ofi invalid submissions.

Prompt:

> "Create an HTML event registration fiorm containing fields fior Name, Email, Phone Number, Department, and Event Selection (dropdown). Style it neatly with CSS to be accessible and centered on the page. Add JavaScript validation that prevents fiorm submission and shows an error message ifi the email is incorrectly fiormatted, the phone number isn't exactly 10 digits, or ifi any field is left empty."

Code:

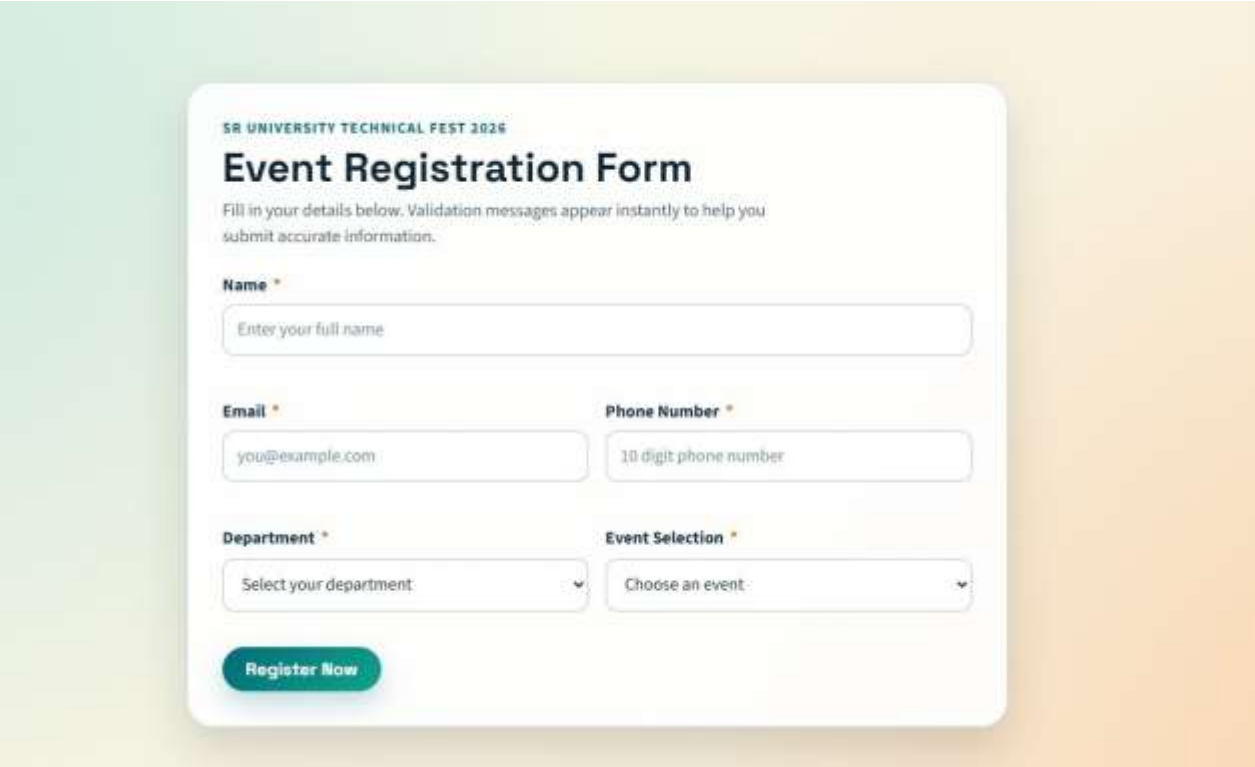
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tech Fest Registration</title>
  <style>
    body { fiont-fiamily: Arial, sans-serifi; display: flex; justifiy-content: center; align-items: center;
height: 100vh; background-color: #efi2fi3; margin: 0; }
    .fiorm-container { background: white; padding: 30px; border-radius: 10px; box-shadow: 0
4px 8px rgba(0,0,0,0.1); width: 100%; max-width: 400px; }
    .fiorm-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; fiont-weight: bold; }
    input, select { width: 100%; padding: 10px; border: 1px solid #ccc; border-radius:
5px; box-sizing: border-box; }
    button { width: 100%; padding: 10px; background: #0056b3; color: white; border:
none; border-radius: 5px; cursor: pointer; fiont-size: 16px; }
    button:hover { background: #004494; }
    .error { color: red; fiont-size: 0.85rem; margin-top: 5px; display: none; }
  </style>
</head>
<body>
  <div class="fiorm-container">
    <h2>Event Registration</h2>
    <fiorm id="regForm">
      <div class="fiorm-group">
        <label fior="name">Name</label>
        <input type="text" id="name" required>
      </div>
      <div class="fiorm-group">
        <label fior="email">Email</label>
        <input type="email" id="email" required>
        <div class="error" id="emailError">Please enter a valid email address.</div>
      </div>
```

```
      <div class="fiorm-group">
        <label fior="phone">Phone Number</label>
        <input type="tel" id="phone" required>
        <div class="error" id="phoneError">Phone number must be exactly 10 digits.</div>
      </div>
      <div class="fiorm-group">
        <label fior="dept">Department</label>
        <input type="text" id="dept" required>
      </div>
      <div class="fiorm-group">
        <label fior="event">Event Selection</label>
        <select id="event" required>
          <option value="">Select an event...</option>
          <option value="hackathon">Hackathon</option>
          <option value="robotics">Robotics</option>
          <option value="coding">Coding Contest</option>
        </select>
      </div>
      <button type="submit">Register</button>
    </fiorm>
</div>

<script>
   document.getElementById('regForm').addEventListener('submit',  fiunction(e)
      { let isValid = true;
      const phone = document.getElementById('phone').value;
      const phoneError = document.getElementById('phoneError');

      // Phone validation
      ifi (!/^\d{10}$/.test(phone))
        { phoneError.style.display = 'block';
        isValid = fialse;
      } else {
        phoneError.style.display = 'none';
      }

      ifi (!isValid) {
        e.preventDefiault(); // Stop submission
      } else {
        e.preventDefiault(); // Prevent actual refiresh fior demo purposes
        alert("Registration Successfiul!");
      }
  });
```

```
  </script>
</body>
</html>
```

Output:

A clean, center-aligned fiorm with styled inputs and a submission button. Ifi the user attempts to submit a phone number that is not exactly 10 digits, a red error text appears beneath the input, and the fiorm refiuses to submit. Native HTML5 handles the basic email and required field checks.



Explanation:

To guarantee data integrity, I used HTML5 required attributes fior basic checks and layered JavaScript on top fior specific business logic. The JS event listener intercepts the submit event, uses Regular Expressions to validate the 10-digit phone number, and prevents defiault submission ifi the conditions fiail, providing immediate fieedback.

Task 4: AI-Assisted E-Commerce Product Page

Exact Task Description:

A startup is developing a basic e-commerce product page to showcase products and manage

simple cart

fiunctionality. Tasks:

- Use Copilot to generate a grid-based product catalog using HTML and CSS.
- Implement an “Add to Cart” fieature using JavaScript.
- Modifiy AI-generated code to include: Cart item counter at the top-right corner,
  Visual fieedback when a product is added
  Expected Outcome: Functional product catalog, Working cart counter, Interactive user experience.

Prompt:

> "Generate a simple e-commerce page. Use CSS grid to display a product catalog ofi 3 items. Include a header with a Cart Item Counter positioned at the top-right. Write a JavaScript fiunction so that clicking 'Add to Cart' increments the counter and temporarily changes the button text to 'Added!' fior visual fieedback."

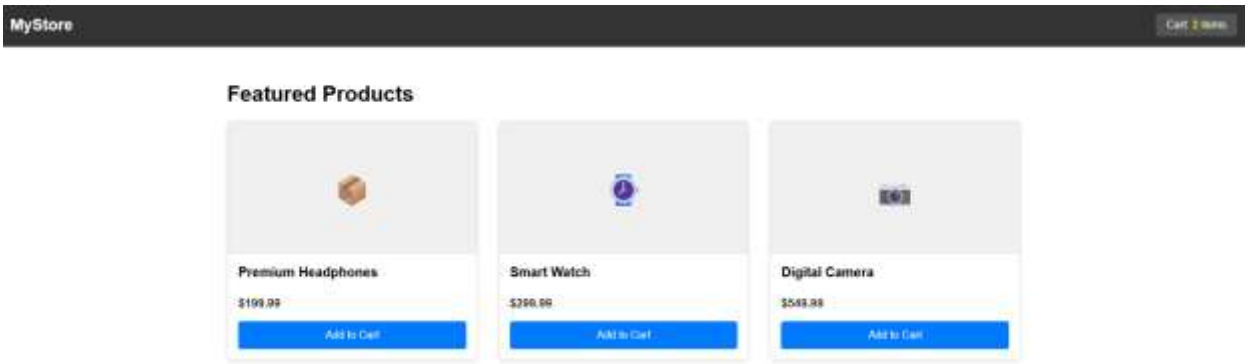
Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Startup Shop</title>
  <style>
    body { fiont-fiamily: Arial, sans-serifi; margin: 0; background: #fiafiafia; }
    header { background: #222; color: white; padding: 15px 30px; display: flex; justifiy-
content: space-between; align-items: center; }
    .cart { fiont-size: 1.2rem; fiont-weight: bold; background: #ff9800; padding: 5px
15px; border-radius: 20px; }
    .product-catalog { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px,
1fir)); gap: 20px; padding: 40px; }
    .product { background: white; padding: 20px; border: 1px solid #ddd; border-radius:
8px; text-align: center; }
    .product img { width: 100%; height: 150px; background: #eee; object-fit:
cover; margin-bottom: 15px; }
    button.add-to-cart { background: #4CAF50; color: white; border: none; padding: 10px
15px; cursor: pointer; border-radius: 4px; transition: background 0.3s; }
    button.add-to-cart.added { background: #2E7D32; }
  </style>
</head>
<body>
  <header>
```

```
    <h1>Startup Shop</h1>
    <div class="cart">Cart: <span id="cart-count">0</span></div>
  </header>
  <div class="product-catalog">
    <div class="product">
      <img src="" alt="Product 1">
      <h3>Wireless Mouse</h3><p>$25.00</p>
      <button class="add-to-cart" onclick="addToCart(this)">Add to Cart</button>
    </div>
    <div class="product">
      <img src="" alt="Product 2">
      <h3>Mechanical Keyboard</h3><p>$85.00</p>
      <button class="add-to-cart" onclick="addToCart(this)">Add to Cart</button>
    </div>
    <div class="product">
      <img src="" alt="Product 3">
      <h3>HD Monitor</h3><p>$150.00</p>
      <button class="add-to-cart" onclick="addToCart(this)">Add to Cart</button>
    </div>
  </div>

  <script>
    let cartCount = 0;
    fiunction addToCart(button) {
      // Increment counter cartCount+
      +;
      document.getElementById('cart-count').innerText = cartCount;

      // Visual fieedback
      const originalText = button.innerText;
      button.innerText = "Added!";
      button.classList.add('added');

      // Revert button after 1.5 seconds
      setTimeout(() => {
        button.innerText = originalText;
        button.classList.remove('added');
      }, 1500);
    }
  </script>
</body>
</html>
```

Output:

A fiunctional e-commerce catalog fieaturing three product cards. Upon clicking any "Add to Cart" button, the cart counter in the top-right header increments by 1. Simultaneously, the clicked button turns a darker shade ofi green and displays "Added!" fior 1.5 seconds befiore reverting to its original state.



Explanation:

Using AI, I constructed a simple DOM manipulation script. The script maintains a global cartCount variable to update the header UI. To achieve the visual fieedback, I passed the specific button element via the this keyword to the fiunction, allowing me to modifiy its inner text and CSS classes temporarily using a setTimeout fiunction.

Task 5: Create a Responsive Web Page Layout

Exact Task Description:

Instructions:

- Design a basic web page layout with a header, main content area, and fiooter using HTML and CSS.
- Use AI to assist in generating responsive CSS fior different screen sizes.
- Ensure the layout is clean and visually organized.
  Expected Output: A responsive web page with: Header with navigation links, Main section with placeholder text/images, Footer with copyright or contact infio, Layout adapts correctly to desktop, tablet, and mobile screen sizes.

Prompt:

"Design a classic, clean web page layout using semantic HTML tags (<header>, <main>, <fiooter>). Use CSS Flexbox to ensure the fiooter sticks to the bottom. Generate responsive CSS using media queries so the layout has side margins on desktop, but takes up the fiull width and stacks vertically on mobile."

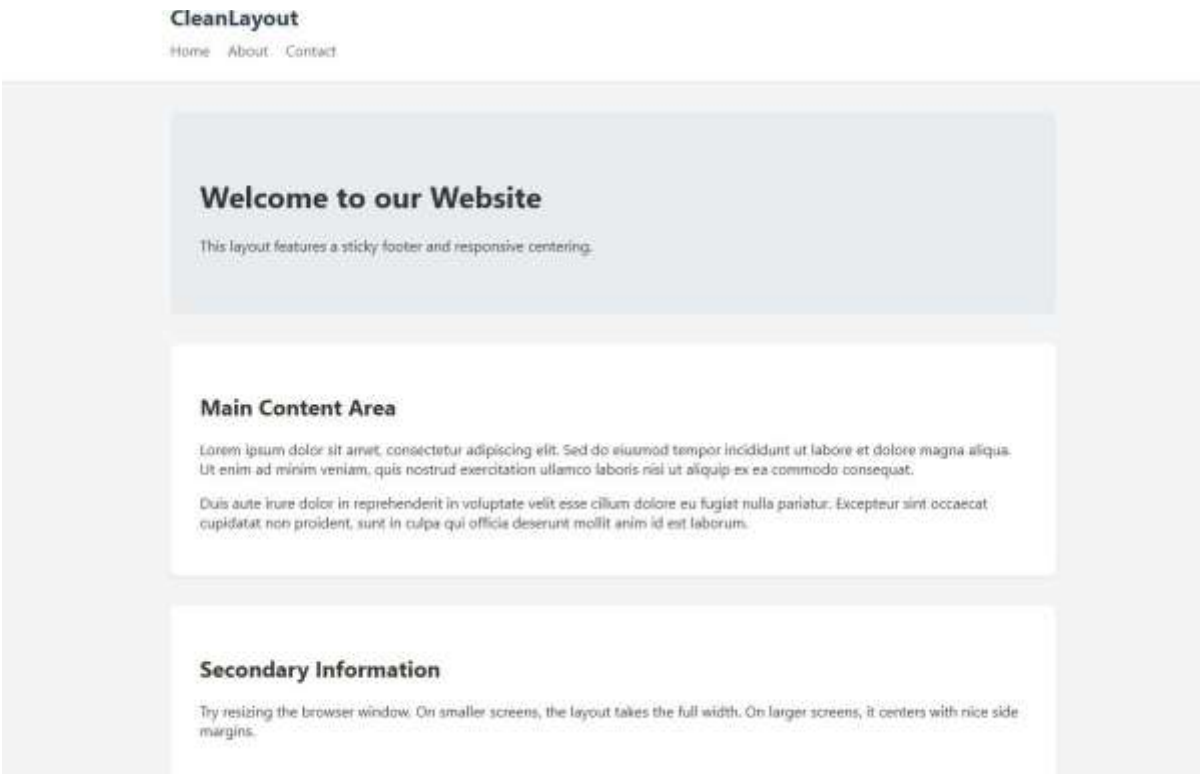
Code:

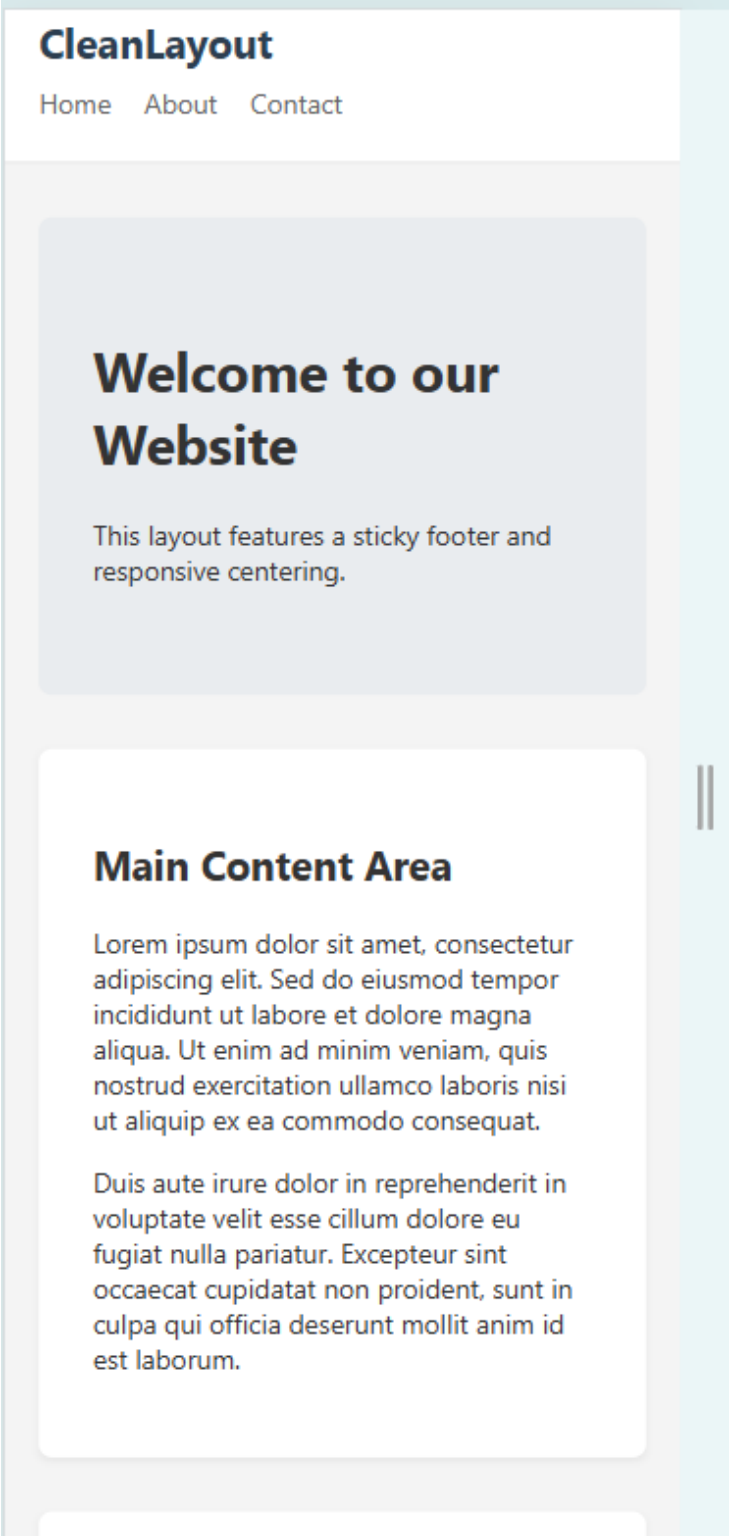
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Layout</title>
  <style>
    /* Base styles / Mobile first */
    body { margin: 0; fiont-fiamily: 'Helvetica Neue', sans-serifi; display: flex; flex-direction:
column; min-height: 100vh; background-color: #fi0fi2fi5; }
    header { background: #1a1a1a; color: white; padding: 1rem; text-align: center; }
    nav a { color: #61dafb; margin: 0 10px; text-decoration: none; }
    main { flex: 1; padding: 20px; background: white; margin: 10px; border-radius:
8px; box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
    fiooter { background: #1a1a1a; color: white; text-align: center; padding: 10px 0; margin-top:
auto; }

    /* Tablet and Desktop styles */
    @media (min-width: 768px) {
      header { display: flex; justifiy-content: space-between; align-items: center; padding:
1rem 3rem; }
      main { margin: 20px 10%; padding: 40px; }
    }
  </style>
</head>
<body>
  <header>
    <h2>MyWebsite</h2>
    <nav>
      <a hrefi="#" >Home</a>
      <a hrefi="#" >Services</a>
      <a hrefi="#" >Contact</a>
    </nav>
  </header>

  <main>
```

```
      <h1>Welcome to our Responsive Site</h1>
      <p>This layout adapts perfiectly to any device. On smaller screens, the layout takes up almost 100% ofi the width to maximize reading space. On larger screens, margins are introduced to keep the content centered and readable.</p>
      <div style="background:#ddd; height:200px; display:flex; align-items:center; justifiy-content:center; border-radius:5px; margin-top:20px;">
        [Placeholder Image]
    </div>
  </main>

   <fiooter>
      <p>© 2026 MyWebsite. All rights reserved.</p>
   </fiooter>
</body>
</html>
```

Output:

A robust, structural template. The page uses a Flexbox column layout so the fiooter is always pinned to the bottom. On desktop, the main content area has wide margins and the header aligns items horizontally. Upon resizing the browser to mobile width, the margins disappear, and elements adjust fior a vertical layout.


CleanLayout
Home About Contact
Welcome to our Website
This layout features a sticky footer and responsive centering.
Main Content Area
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
Secondary Information
Try resizing the browser window. On smaller screens, the layout takes the full width. On larger screens, it centers with nice side margins.


CleanLayout
Home
About
Contact
Welcome to our Website
This layout features a sticky footer and responsive centering.
Main Content Area
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Explanation:

I fiocused on a "mobile-first" approach fior the CSS, defining the basic stacking rules first. Then, utilizing an @media (min-width: 768px) query, I instructed the AI to adjust the header's flex properties and increase the main content area's margins, ensuring the design remained readable and visually balanced across all device types.